COMPARISON

wkhtmltopdf alternatives, and what changes when you switch

Updated 19 Jan 2027

KEY TAKEAWAYS

- wkhtmltopdf's GitHub repository was archived on 2 January 2023, and its downloads page names 0.12.6, released on 11 June 2020, as the current stable series.
- Its own status page suggests WeasyPrint or Prince for reports from HTML you control, and Puppeteer for sites with dynamic JavaScript.
- Headless Chromium (Playwright, Puppeteer, Gotenberg) uses a current browser engine; WeasyPrint has its own engine and runs no JavaScript; Prince is commercial.
- A different engine means different page breaks, fonts and line wraps. Print with both tools and compare the page counts before you switch.
- wkhtmltopdf's header and footer options have a counterpart in each tool, with different syntax.

CONTENTS

1 Why people move off wkhtmltopdf 2
2 wkhtmltopdf alternatives side by side 3
3 Headless Chromium: Playwright, Puppeteer, Gotenberg 4
4 WeasyPrint 5
5 Prince 5
6 Moving over: what usually changes 7
7 Where papyrus fits 7
8 What to remember 8

wkhtmltopdf's own pages say where the project stands. On 11 November 2026, its GitHub page said: "This repository was archived by the owner on Jan 2, 2023. It is now read-only." Its downloads page said: "The current stable series is 0.12.6, which was released on June 11, 2020". If you need a wkhtmltopdf alternative, the main replacements are headless Chromium (through Playwright, Puppeteer or Gotenberg), WeasyPrint and Prince.

The project's status page, whose figures are dated June 2020, points the same way. For reports from HTML you control, it suggests WeasyPrint or the commercial Prince. For sites with dynamic JavaScript, it suggests Puppeteer or one of its wrappers. Below are those options, how WeasyPrint and Playwright differ, and what changes when you move. papyrus and n-uf are not affiliated with Microsoft (Playwright), Google (Puppeteer, Chrome), the Gotenberg project, Kozea and CourtBouillon (WeasyPrint), YesLogic (Prince) or the wkhtmltopdf project.

1. Why people move off wkhtmltopdf

These are the reasons its own pages give.

- **The engine.** Its home page says it renders with the Qt WebKit rendering engine. The status page's summary says the Qt 4 it uses hasn't been supported since 2015, and the WebKit in it hasn't been updated since 2012. It also explains that QtWebKit was deprecated in 2015 and removed in 2016.
- **The release.** The current stable series is 0.12.6, from June 2020, and the repository is archived.
- **The warning.** The downloads page says not to use wkhtmltopdf with any untrusted HTML, and that unsanitized user HTML can lead to a complete takeover of the server it runs on. The status page adds a mandatory access control system such as AppArmor or SELinux.
- **The advice.** The status page itself names WeasyPrint, Prince and Puppeteer.

None of this says wkhtmltopdf's output is wrong. If it prints HTML you control and you're happy with the result, it won't stop working the day you read this. The question is what you build on next.

2. wkhtmltopdf alternatives side by side

	wkhtmltopdf	Playwright, Puppeteer	Gotenberg	WeasyPrint	Prince
Engine	Qt WebKit	Chromium (Playwright's `page.pdf()` works only in Chromium); Puppeteer drives Chrome	Chromium; LibreOffice for office files	Its own, written in Python; not WebKit or Gecko	Its own formatter
How you call it	Command line; a C library	A Node library (Playwright also has a Python version)	HTTP API with `multipart/form-data`; its docs list a PHP SDK	Command line and Python library	Command line; packages or libraries for Python, Node.js, Java, .NET, PHP and Ruby on Rails
Paged-media CSS	Not described on the pages we read; headers and footers through its own options	What Chromium supports: `@page` size, and margin boxes from Chrome 131	As Chromium, plus `header.html` and `footer.html`	Its docs say all of Paged Media Level 3 is available, margin boxes and page counters included	Page size, page regions such as headers and footers, page numbering
JavaScript	Yes; can be turned off	Yes	Yes (Chromium)	No	Off by default; can be enabled
Licence	LGPLv3	Apache-2.0	MIT	BSD	Commercial; free version adds a watermark
Runs on	Builds for Windows, macOS and several Linux distributions	Wherever the browser runs	A Docker container	Python 3.10 or later and Pango (WeasyPrint 70.0); packaged for Debian, Ubuntu, Fedora and others	Windows, macOS, Linux and FreeBSD

The JavaScript cell for wkhtmltopdf comes from its manual, which has `--javascript-delay` and `--disable-javascript`. Every other cell is from that project's own pages, listed in the sources and read on 11 November 2026.

3. Headless Chromium: Playwright, Puppeteer, Gotenberg

This is the route furthest from wkhtmltopdf's engine, and the wkhtmltopdf alternative its status page points to for scripted pages. You get the engine of a current browser. Playwright's `page.pdf()` prints with the page's print CSS; Puppeteer does the same job with its own API. The code is in Playwright: generate PDF from HTML, and the Node routes are compared in convert html to pdf nodejs.

Where it fits: the pages are web pages, with scripts, current CSS and fonts the browser loads. MDN's data lists `@page` margin boxes from Chrome 131, so headers and footers have a CSS route as well as the template route.

Where it doesn't: you install and update a browser. Playwright installs Chromium with `npx playwright install chromium`, and Puppeteer downloads Chrome with the package unless you use `puppeteer-core`. And if your app is written in PHP, neither library runs inside it; Gotenberg puts the same engine behind HTTP.

Gotenberg

Gotenberg's README calls it a Docker-based API for PDF. Its docs list Chromium routes for web pages, HTML and Markdown, and LibreOffice routes for office documents. You POST files as `multipart/form-data` and get a PDF back:

```
docker run --rm -p 3000:3000 gotenberg/gotenberg:9
curl --request POST http://localhost:3000/forms/chromium/convert/html \
  --form files=@index.html -o out.pdf
```

We haven't run it for this article; the commands come from its docs.

Where it fits: the app that makes the PDF isn't Node. Any language that can send a form can use it, and its docs list a PHP SDK. Headers and footers are `header.html` and `footer.html` files, where elements with the classes `pageNumber` and `totalPages` get the values: the same idea as wkhtmltopdf's `[page]` and `[topage]`.

Where it doesn’t: you now run a service. Its docs have a guide to securing a deployment, and the project is on version 9 with the 8.x docs kept separately, so check which version an example is for. Chromium prints the page as soon as it loads, so a page that draws later needs one of the wait fields.

4. WeasyPrint

WeasyPrint’s README says it turns simple HTML pages into reports, invoices and tickets. It’s a visual rendering engine for HTML and CSS that exports to PDF, and free software under a BSD licence. It isn’t based on a full rendering engine like WebKit or Gecko; its CSS layout engine is written in Python. The command line is `weasyprint input.html output.pdf`, and in Python it’s `HTML(...).write_pdf()`. WeasyPrint 70.0, the release its docs cover, needs Python 3.10 or later and Pango 1.44 or later.

Its docs say all the features of CSS Paged Media Level 3 are available: the `@page` rule and its selectors, margin boxes, page counters (with known limitations), page size, bleed and marks, and named pages. From the generated-content module it lists running elements, `string-set` and footnotes. Chromium doesn’t have `string-set` yet: Chrome’s 2024 post lists it as a future possibility, and the installed Chrome 157 ignored it when we tried it on 11 November 2026.

Where it fits: documents from HTML you control, like the README’s reports, invoices and tickets, in a Python app, where print layout matters more than matching a browser. It’s also the first tool wkhtmltopdf’s status page names for that job.

Where it doesn’t: its docs say there is no JavaScript, no user interaction and no live rendering. It prints warnings on standard error for CSS properties it doesn’t support, so a page designed for a browser needs checking. And its docs warn that untrusted HTML or CSS may lead to security problems.

5. Prince

Prince’s home page describes Prince 17 as a way to convert HTML to PDF, with headers and footers, page numbers and bookmarks. It reads HTML, Markdown, XML, SVG and CSS, and offers Python and Node.js packages, Java and .NET libraries, PHP and Ruby on Rails integrations, and Docker images. Its docs say JavaScript isn’t run by default and has to be enabled.

It is commercial. On 11 November 2026, its purchase page listed:

- a Commercial Site License for any number of servers, for an annual fee with upgrades and support, starting around USD 2,000 a year for startups and depending on the type and volume of documents;
- Per Server Licensing for government institutions and other noncommercial entities that produce documents for internal use or publish them free of charge: a one-time fee of USD 3,800 for one server, with 12 months of upgrades and support;
- a Desktop License at USD 495 for one user on one computer.

The EULA's non-commercial licence puts notices into the generated PDFs and excludes documents such as invoices and receipts. The installation guide says a purchased licence file removes the watermark on generated documents, and that the free version needs no licence file.

Where it fits: print-first documents where paged-media features and a vendor with support count, and you can license it.

Where it doesn't: commercial use needs a licence, and the free version marks its output. Read the purchase page and the EULA, not this summary, before you decide.

6. Moving over: what usually changes

A different engine can paginate the same HTML differently. A checklist, in the order that tends to save time:

1. **Print with both and compare.** The same HTML through the old tool and the new one; count the pages. A one-page difference is the first sign that breaks moved.
2. **Page breaks.** Re-check every forced and avoided break. css page break print lists the properties and the cases where they don't apply.
3. **Paper and margins.** wkhtmltopdf's `--page-size` becomes `@page { size }` in CSS, `format` and `margin` in Playwright, or `paperSize` and the margin fields in Gotenberg.
4. **Headers and footers.** wkhtmltopdf's `--header-html` and `--footer-html` take variables such as `[page]` and `[topage]`. Playwright's templates and Gotenberg's `header.html` and `footer.html` use the classes `pageNumber` and `totalPages`. WeasyPrint and Prince use page margin boxes and counters in CSS. css print header and footer and css print page numbers cover the CSS side.
5. **JavaScript timing.** `--javascript-delay` waits a fixed time. Gotenberg has `waitDelay`, `waitForSelector` and `waitForExpression`, and a Playwright script can wait for an element or for `document.fonts.ready`. WeasyPrint runs no JavaScript at all.
6. **Fonts.** Text metrics change with the engine, so line wraps and page breaks can too. Install the fonts, load them with `@font-face`, and look at the output.
7. **Untrusted HTML.** Keep sanitizing it, whichever tool you move to. WeasyPrint's docs warn about it, as do wkhtmltopdf's.

7. Where papyrus fits

papyrus is a toolkit by n-uf for documents rendered as HTML and exported as PDF. It isn't a drop-in for wkhtmltopdf, which converts any page you give it. papyrus is for documents laid out as fixed sheets: each element with the class `pdf-page` is one sheet on screen and one page in the PDF, and it prints them with headless Chromium through Playwright. Before it writes a PDF, it checks that the page elements, the PDF's page count and the expected `pageCount` agree. That's step 1 of the checklist, done on every export.

If your documents have that shape (a report, a résumé, an invoice with a known number of pages), the `papyrus pdf` command takes a config and prints them, and plain HTML works from a `file:` URL with no server. The **html to pdf a4 page** has the details.

papyrus is young. Its versions are dates (`YY.M.R`), not semantic versions, so any release may break things. Pin an exact version and read the CHANGELOG before you upgrade.

8. What to remember

- wkhtmltopdf's repository is archived, and its stable series dates from June 2020. Its own pages suggest WeasyPrint, Prince or Puppeteer.
- Chromium (Playwright, Puppeteer, Gotenberg) for pages as a browser shows them; WeasyPrint for HTML you control; Prince if you can license it.
- Print with the old tool and the new one side by side. Page breaks, fonts and headers are where the differences show.

Questions people ask

Is wkhtmltopdf still maintained?

Its GitHub repository was archived on 2 January 2023 and is read-only, and its downloads page says the current stable series is 0.12.6, released on June 11, 2020. The project's status page says the Qt 4 it uses hasn't been supported since 2015. Read those pages before you build anything new on it.

What is the best wkhtmltopdf alternative on Linux?

For pages as a browser shows them, headless Chromium through Playwright or Puppeteer, or Gotenberg if you want it as a Docker service. For HTML you control, WeasyPrint is packaged for Debian, Ubuntu, Fedora and other distributions. Prince runs on Linux and FreeBSD and is sold under licence.

WeasyPrint vs Playwright: which should I use?

Playwright if you need a browser's rendering or JavaScript, WeasyPrint if you control the HTML and want paged-media CSS without a browser. WeasyPrint's docs say it runs no JavaScript and isn't based on a full rendering engine such as WebKit or Gecko. Playwright prints with Chromium, so it supports whatever Chromium does.

Is there a free Prince XML alternative?

Yes: WeasyPrint is free software under a BSD licence, and its docs list CSS Paged Media features, page margin boxes and page counters included. Prince itself has a free version; its installation guide says a purchased licence file removes the watermark on generated documents.

Will my wkhtmltopdf output look the same after I switch?

Not exactly. wkhtmltopdf renders with Qt WebKit, and each alternative uses a different engine, so page breaks, fonts and line wraps can move. Print the same HTML with both tools and compare the page counts and break points before you switch.

Sources

1. wkhtmltopdf: home page ↗ checked 11 Nov 2026
2. wkhtmltopdf: project status ↗ checked 11 Nov 2026
3. wkhtmltopdf: downloads ↗ checked 11 Nov 2026
4. GitHub: wkhtmltopdf/wkhtmltopdf ↗ checked 11 Nov 2026
5. wkhtmltopdf: usage manual (0.12.6) ↗ checked 11 Nov 2026
6. Playwright: Page, page.pdf() ↗ checked 11 Nov 2026
7. Playwright: PDF Export (MCP tool) ↗ checked 11 Nov 2026
8. Playwright: Library ↗ checked 11 Nov 2026
9. Playwright for Python: Installation ↗ checked 11 Nov 2026
10. Puppeteer: home page ↗ checked 11 Nov 2026
11. npm registry: playwright and puppeteer ↗ checked 11 Nov 2026
12. MDN browser-compat-data 8.1.8 (npm, the latest on 2026-11-11) ↗ checked 11 Nov 2026
13. Chrome for Developers: Add content to the margins of web pages when printed using CSS (30 October 2024) ↗ checked 11 Nov 2026
14. Gotenberg README ↗ checked 11 Nov 2026
15. Gotenberg docs: introduction (9.x) ↗ checked 11 Nov 2026
16. Gotenberg: Convert HTML to PDF (9.x) ↗ checked 11 Nov 2026
17. WeasyPrint README ↗ checked 11 Nov 2026
18. WeasyPrint 70.0 documentation: API reference ↗ checked 11 Nov 2026
19. WeasyPrint 70.0 documentation: First steps ↗ checked 11 Nov 2026
20. WeasyPrint 70.0 documentation: Going further ↗ checked 11 Nov 2026
21. Prince: home page ↗ checked 11 Nov 2026
22. Prince: purchase page ↗ checked 11 Nov 2026
23. Prince: end user licence agreement ↗ checked 11 Nov 2026
24. Prince documentation: Installing Prince ↗ checked 11 Nov 2026
25. Prince documentation: Paged media ↗ checked 11 Nov 2026
26. Prince documentation: JavaScript ↗ checked 11 Nov 2026
27. papyrus README (@n-uf/papyrus v26.11.1) ↗ checked 11 Nov 2026